Практическая работа 12

Проверка полноты и качества аналитической витрины

Вы проверите сохранённую витрину продаж по пятиминутным окнам. Задача — решить, можно ли публиковать результат, и подтвердить решение проверками. Программа уже написана; ваша работа — прогноз, ручная сверка, интерпретация протокола и самостоятельное исправление двух ячеек.

На предыдущем занятии корректировка вернула запоздавшую покупку E009 и изменила общий итог с 4800 на 5500. Теперь правильной общей суммы недостаточно: нужно проверить каждое окно, ключи, типы данных, редакции и связь с исходником. Используются те же 12 событий; переносить старые файлы не требуется.

Работайте индивидуально. Объяснение занимает 25 минут, самостоятельная работа — 60, итоговый разбор — 5. Результаты сохраните локально; загрузите в ЭОС позже, когда преподаватель сообщит ссылку и срок. За эту пару выставляется отдельная оценка 2, 3, 4 или 5.

Нужны Windows 10/11 и Python 3.10 или новее, без дополнительных библиотек. Это локальная модель контроля качества конечного файла. Она не измеряет скорость кластера и не доказывает полноту реального непрерывного потока.

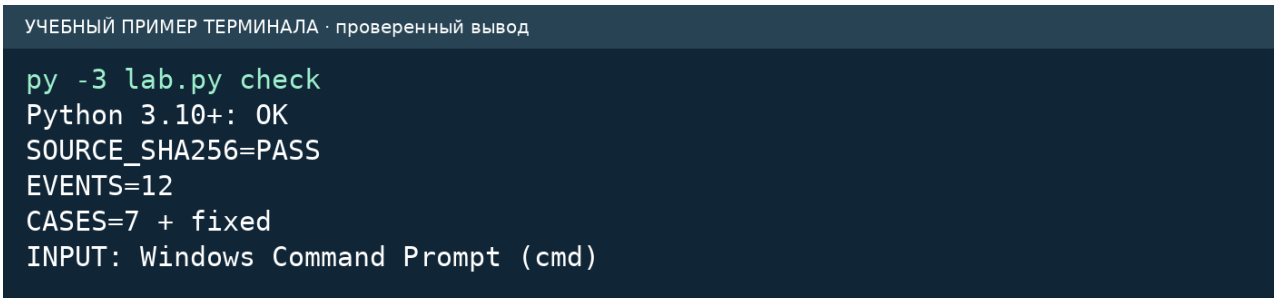
Иллюстрации команд в методичке воспроизводят проверенный вывод программы. Это учебные изображения терминала, а не снимки рабочего стола Windows. Команды приведены также текстом для копирования.

Что записать и проверить

К концу работы: семь решений по примерам, исправленный CSV, повторная проверка, собственный отчёт и ZIP для сдачи.

1 Подготовка папки и командной строки

Скачайте 03_Стартовый_проект.zip. В Проводнике выберите «Извлечь всё». Откройте вложенную папку bigdata12: в ней видны lab.py, REPORT.txt, папки data и cases. Используйте новую папку, чтобы не перезаписать предыдущую работу.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py check
```

Щёлкните адресную строку Проводника именно в этой папке, введите cmd и нажмите Enter. Откроется командная строка Windows. Вводите команды по одной, нажимая Enter. Не копируйте приглашение C:\...>. Не вводите команды в Блокнот или в окно Python с символами >>>.

Выполните команду под рисунком. Альтернатива — двойной щелчок по 00_CHECK.cmd. Оба способа запускают один этап; выполнять его дважды не обязательно. CMD-файлы сами переходят в папку проекта и оставляют окно открытым до нажатия клавиши.

Если py не найден, попробуйте python lab.py check. Если Python отсутствует, откройте DOWNLOADS.html и официальный сайт. Пока среда недоступна, выполните ручную часть; техническую часть необходимо завершить отдельно. Не отмечайте её выполненной заранее.

Что записать и проверить

Проверьте Python, SOURCE_SHA256=PASS и EVENTS=12. При ERROR остановитесь и прочитайте причину.

2 Источник и границы проверки

Откройте data/arrivals.csv через «Открыть с помощью» → Блокнот. Не изменяйте его. В каждой строке есть ID, время события, время поступления, тип, статус и сумма в рублях. Строки описывают один условный день и один поток.

Учитываются только event_type=purchase и status=success. E004 — просмотр, E006 — неуспешная покупка: их суммы не включаются. В этом аудите все успешные покупки известного файла входят в эталон, включая E009. Повторная обработка watermark здесь не выполняется.

Окна полуоткрытые: [10:00,10:05), [10:05,10:10), [10:10,10:15), [10:15,10:20). Левая граница входит, правая нет. E003 в 10:05 относится ко второму окну; E010 в 10:10 — к третьему.

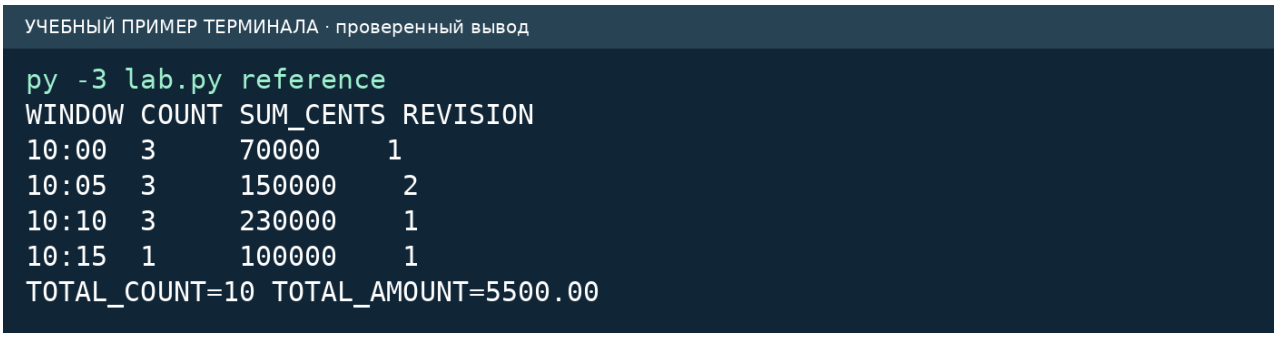
Источник сохранён с SHA-256 в source_manifest.json. Проверка защищает от случайного изменения этих учебных байтов. Она не подтверждает, что реальные события не были потеряны до создания файла. Манифест в этой работе принят как доверенная исходная точка.

Что записать и проверить

В REPORT.txt, пункт 1, выпишите ID покупок каждого окна. Предскажите количество строк витрины до запуска reference.

3 Ручной эталон и редакции

Выполните reference или откройте 01_REFERENCE.cmd. Эталонные количества и суммы вычисляются по исходным событиям. Проверьте их вручную: первое окно 100+200+400; второе 300+500+700; третье 600+800+900; четвёртое 1000.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py reference
```

Поле sum_cents хранит копейки: 70000 означает 700 рублей. Не заменяйте его на 700. Итог: 10 успешных покупок на 5500 рублей. Количество строк витрины равно четырём, а не десяти: одна строка представляет окно.

Ожидаемая редакция для 10:05 равна 2: это результат предыдущей корректировки. У остальных окон редакция 1. Номер редакции не вычисляется из арифметики покупок; здесь это явно заданное условие принятого снимка.

В реальной системе текущую редакцию получают из доверенного журнала или каталога. Меньшая редакция при верных числах в нашем опыте всё равно нарушает договорённость о снимке. Новая большая редакция также потребовала бы согласования, а не автоматического доверия.

Что записать и проверить

Запишите четыре суммы в копейках, четыре количества и ожидаемые редакции. Объясните различие между числом событий и строк витрины.

4 Семь проверок перед публикацией

Каждый CSV в cases проверяется независимо. Рядом лежит одноимённый JSON с source_sha256. Менять исходные примеры нельзя. Рабочую копию fixed.csv вы исправите отдельно.

SCHEMA проверяет точные заголовки, структуру строк, формат окна, неотрицательные целые count и sum_cents, положительную revision. UNIQUE проверяет отсутствие повторов ключа window_start. COVERAGE проверяет совпадение набора ключей с четырьмя ожидаемыми окнами.

TOTALS сопоставляет общее количество и сумму с 10 и 550000 копеек. VALUES сравнивает count и sum_cents каждого окна. REVISION проверяет ожидаемую редакцию каждой строки. SOURCE сопоставляет хеш, указанный в JSON витрины, с манифестом источника.

PASS означает выполнение условия, FAIL — нарушение, SKIP — проверка не выполнялась из-за недопустимых предпосылок. Например, при дубликате нельзя однозначно выбрать строку для сравнения по ключу. SKIP не считается успехом.

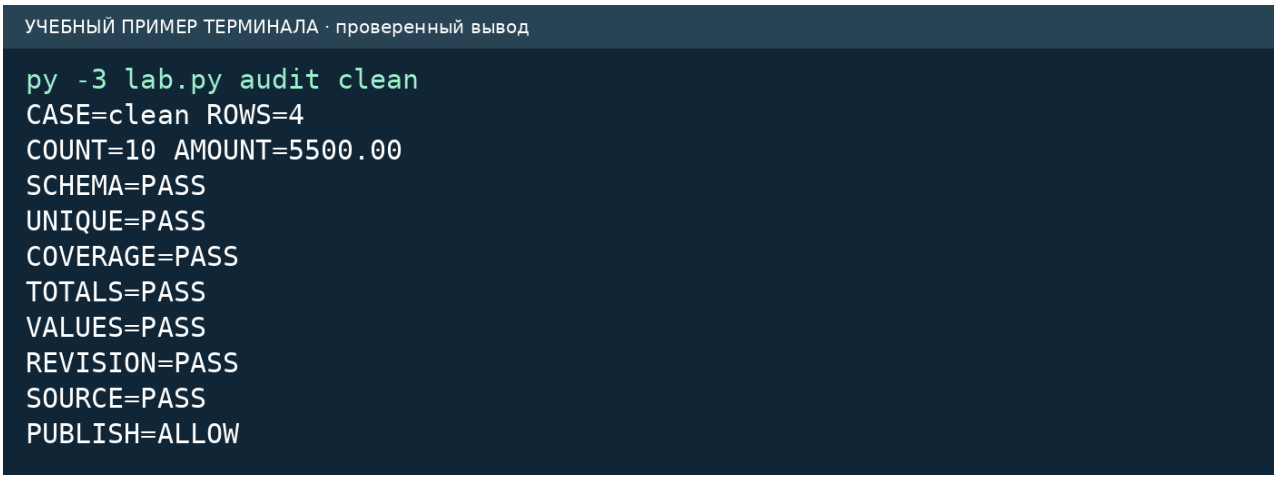
PUBLISH=ALLOW выводится только при семи PASS. Любой FAIL или SKIP приводит к BLOCK. BLOCK — ожидаемый результат аудита плохих данных, а не поломка Python. Код завершения программы 0 означает, что аудит выполнен.

Что записать и проверить

Нарисуйте в отчёте таблицу: пример → нарушенные проверки → решение → объяснение. Заполняйте после каждого запуска.

5 Проверка исправного снимка

Запустите 02_AUDIT_CLEAN.cmd или команду ниже. Откройте results/clean.json. Он содержит решения, подробности и контрольные суммы именно проверенных CSV и JSON.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py audit clean
```

Проверьте четыре строки, десять покупок, сумму 5500 и все семь PASS. Затем откройте cases/clean.csv. Сопоставьте вторую строку окна 10:05 с ручным расчётом и revision=2.

Сохранённый JSON нужен для воспроизводимости: после изменения проверенного файла прежний отчёт теряет актуальность. Команды summary и pack сопоставляют аудит с текущими файлами, а не просто проверяют наличие JSON.

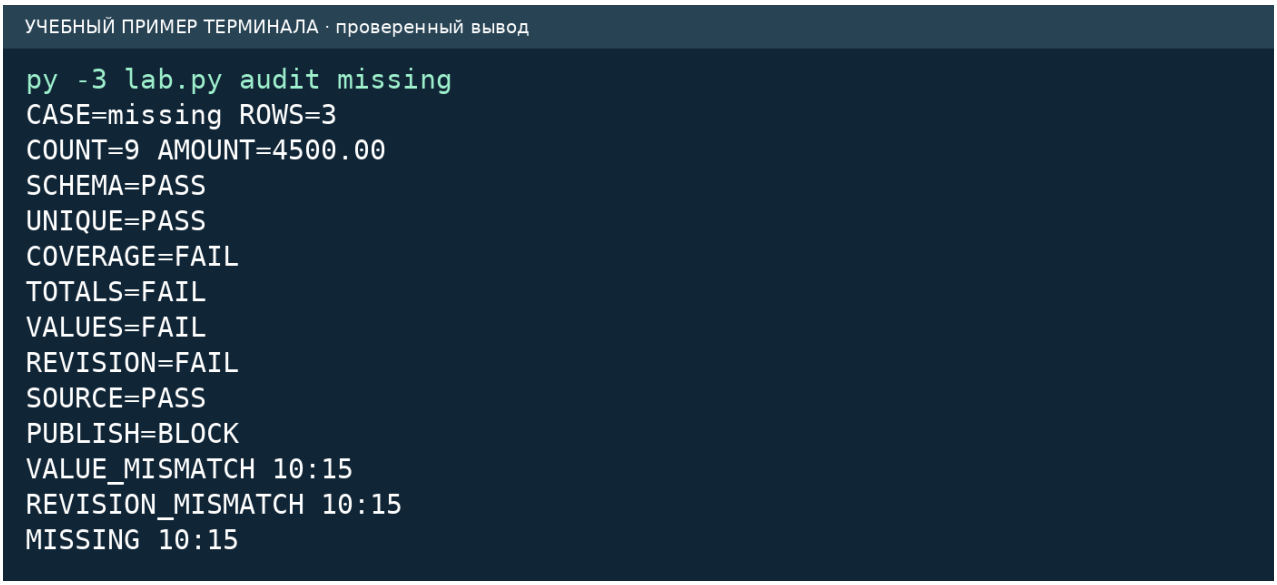
Не используйте clean.csv как замену собственной работы: его цель — показать нормальный протокол. Исправлять нужно fixed.csv, объяснив изменение каждой ячейки на основе событий.

Что записать и проверить

Запишите, что именно позволяет ALLOW. Фраза «всё зелёное» недостаточна: назовите семь условий.

6 Пропущенное окно

До запуска спрогнозируйте итог, если из витрины исчезнет окно 10:15 с одной покупкой на 1000. Запишите прогноз в пункт 2 отчёта. Затем выполните audit missing или 03_AUDIT_MISSING.cmd.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py audit missing
```

Найдите строку MISSING 10:15. COVERAGE обнаруживает отсутствие ожидаемой группы. VALUES и REVISION тоже не могут подтвердить полный снимок: ожидаемой строки нет. Итоговые количество и сумма отличаются от эталона.

Откройте cases/missing.csv, посчитайте строки и проверьте сумму вручную. Нельзя объявлять отсутствие строки нулевыми продажами: в источнике есть E012. Если окно действительно пустое, правило его представления должно быть определено отдельно.

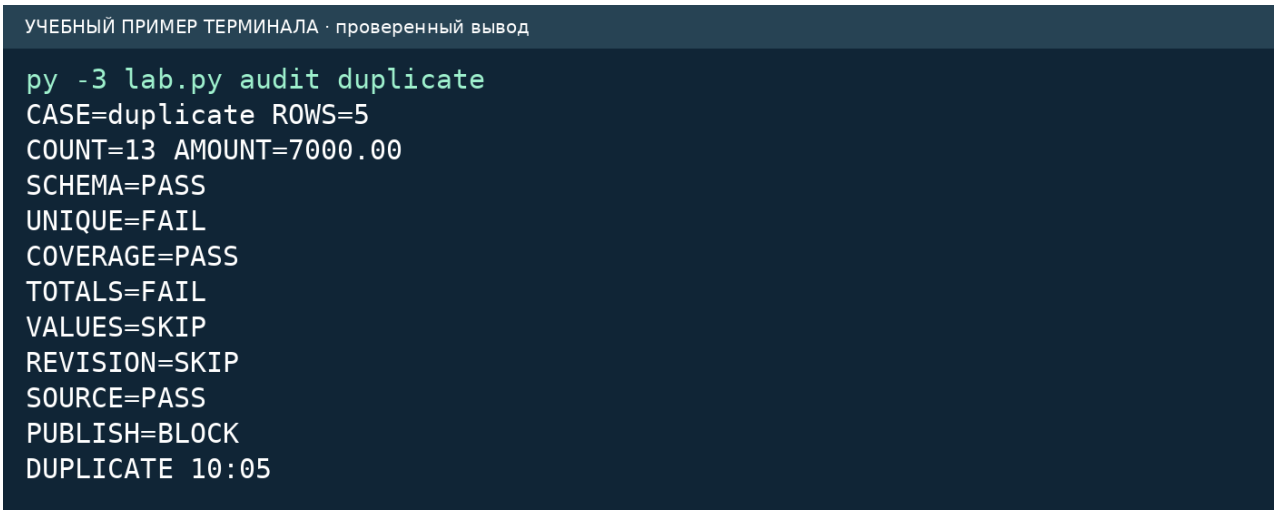
В текущем проекте обязательны окна, в которых есть успешные покупки известного источника. Он не требует заполнить все 288 пятиминутных окон суток. Это существенная граница проверки полноты.

Что записать и проверить

Запишите прогноз и факт: число покупок, сумма, отсутствующий ключ и решение о публикации.

7 Повтор ключа

В duplicate повторена строка окна 10:05. Перед запуском вычислите, как изменятся количество покупок и общая сумма, если обе строки сложить. Выполните audit duplicate или 04_AUDIT_DUPLICATE.cmd.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py audit duplicate
```

UNIQUE=FAIL обнаруживает повтор ключа, хотя формат обеих строк корректен. COVERAGE может остаться PASS: все нужные ключи присутствуют, но один встречается дважды. Поэтому проверки уникальности и покрытия нужны отдельно.

VALUES и REVISION получают SKIP. Программа не выбирает произвольно первую либо последнюю строку. Нельзя считать это доказательством корректности значений. На этом этапе публикация блокируется.

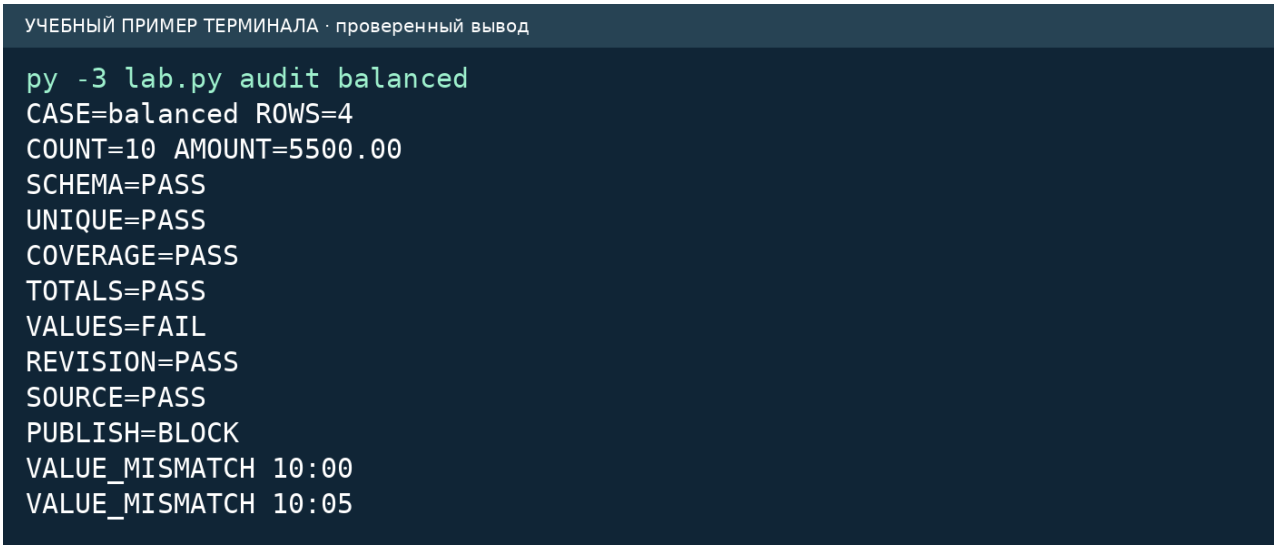
Удаление повторной строки допустимо только после анализа происхождения. В учебном примере повтор идентичен, но в реальных данных одинаковый ключ может сопровождаться разными суммами и редакциями. Автоматическое удаление любой строки может потерять корректное обновление.

Что записать и проверить

Объясните, почему COVERAGE=PASS и UNIQUE=FAIL не противоречат друг другу. Сравните прогноз и фактический итог.

8 Компенсирующие ошибки сумм

В balanced четыре уникальных ключа и корректные типы. Запустите audit balanced или 05_AUDIT_BALANCED.cmd. Найдите сочетание TOTALS=PASS и VALUES=FAIL.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py audit balanced
```

Откройте cases/balanced.csv. В окне 10:00 записано 80000 вместо 70000 копеек; в 10:05 — 140000 вместо 150000. Ошибки +10000 и −10000 копеек компенсировались. Общая сумма и количество покупок совпали с эталоном.

Распределение продаж по времени при этом неверно. Руководитель, сравнивающий интервалы, получит ошибочный вывод, даже если итоговый оборот дня правильный. Контроль только одной итоговой суммы пропустит такой дефект.

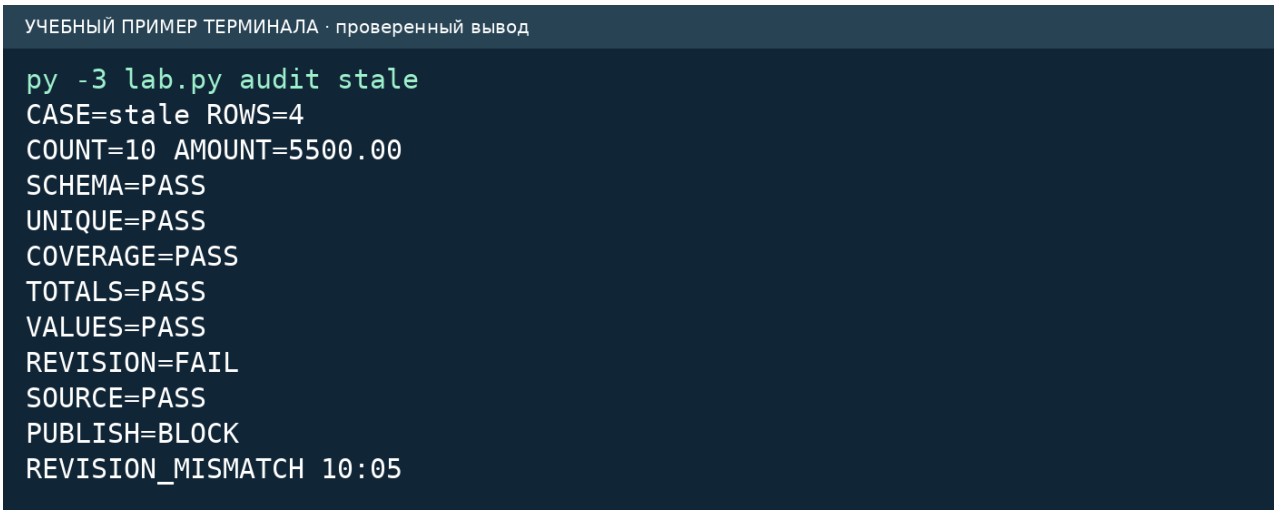
В отчёте приведите обе строки: значение витрины, ожидаемое значение и разницу «витрина минус эталон». Не меняйте balanced.csv: это исходный дефектный пример, который должен остаться воспроизводимым.

Что записать и проверить

Рассчитайте две разницы в копейках и рублях. Объясните, какая проверка выявила проблему при совпавшем TOTALS.

9 Проверка редакции и связи с источником

Выполните audit stale или 06_AUDIT_STALE.cmd. Суммы и количества правильные, но для 10:05 стоит revision=1 вместо принятой 2. REVISION=FAIL блокирует публикацию в рамках данного договора о снимке.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py audit stale
```

Затем выполните py -3 lab.py audit foreign или 08_AUDIT_FOREIGN.cmd. Этот пример содержит правильный CSV, но в cases/foreign.json указан другой source_sha256. SOURCE=FAIL означает отсутствие согласованной связи с известным исходником.

Нельзя просто переписать хеш или редакцию ради PASS. Для реального исправления необходимо подтвердить происхождение и историю расчёта. Здесь достаточно обнаружить нарушение и предложить обоснованное действие: запросить правильный снимок или пересчитать из доверенного источника.

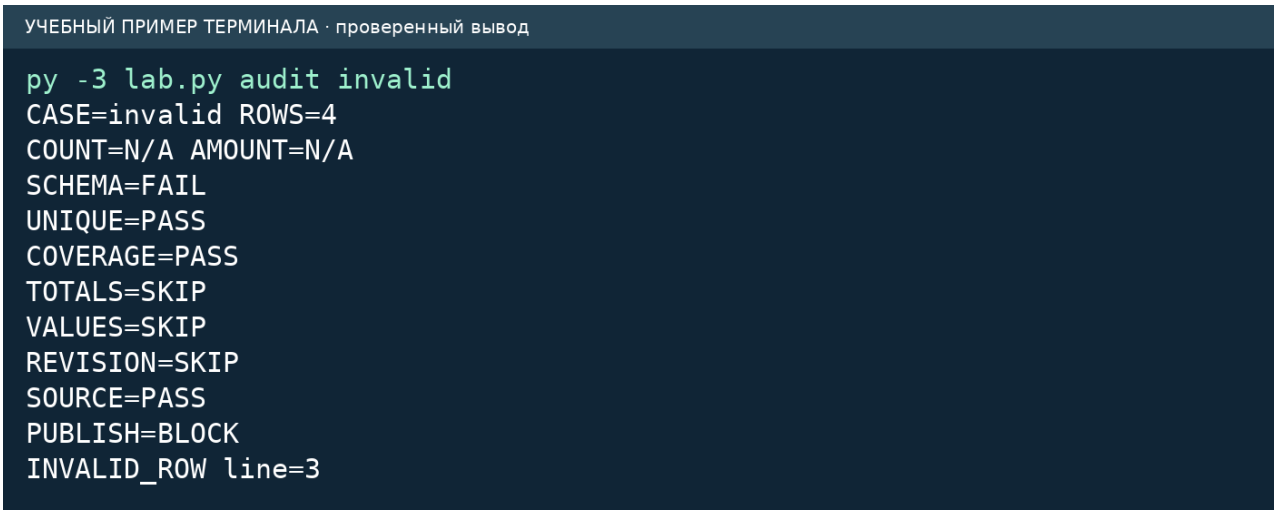
Хеш в метаданных сам по себе не доказывает, что программа действительно считала по этому файлу. Наш аудит проверяет заявленную связь и точность чисел относительно доступного источника. Подлинность происхождения требует дополнительных мер за пределами опыта.

Что записать и проверить

Сравните stale и foreign: какое условие нарушено в каждом, почему одного совпадения чисел недостаточно?

10 Неправильный тип и зависимые проверки

Выполните audit invalid или 07_AUDIT_INVALID.cmd. В поле count окна 10:05 записано three вместо целого числа. SCHEMA=FAIL сопровождается INVALID_ROW с номером строки CSV; заголовок считается первой строкой.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py audit invalid
```

Проверка итогов показывает SKIP, а общие значения N/A: программа не подменяет неправильную запись нулём и не выдаёт сумму только по распознанной части за итог всей витрины. Сравнение значений и редакций также пропущено.

UNIQUE и COVERAGE можно проверить по доступным ключам, поэтому они остаются PASS. Это не снимает ошибку схемы. Итоговое решение BLOCK учитывает все условия, а не большинство успешных проверок.

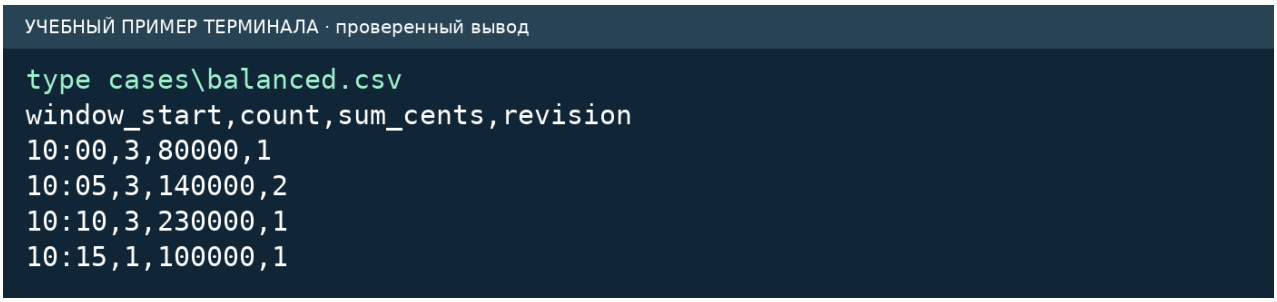
В отчёте различите три состояния: выполнено и верно; выполнено и неверно; не выполнено. Непроверенное условие нельзя считать выполненным. Исправление типа должно опираться на источник: для этого окна ожидается count=3.

Что записать и проверить

Почему замена three на 0 была бы ложным исправлением? Что должен сделать обработчик вместо молчаливого преобразования?

11 Самостоятельное исправление рабочей копии

cases/fixed.csv изначально повторяет дефект balanced. Только этот CSV разрешено изменять. Откройте его командой notepad cases\fixed.csv в CMD либо через «Открыть с помощью» → Блокнот. Измените ровно два значения sum_cents.



Учебная иллюстрация команды и проверенного вывода

```
type cases\balanced.csv
```

Для окна 10:00 восстановите сумму трёх его покупок, для 10:05 — сумму E003, E007, E009. Не меняйте count, window_start, revision и заголовок. Сохраните Ctrl+S и закройте редактор. Проверьте, что файл не превратился в fixed.csv.txt.

В этой работе исправляются дефекты учебной копии принятого снимка, поэтому сохраняются его редакции 1/2/1/1. Это не пример выпуска новой производственной редакции. В настоящем процессе потребовалась бы запись изменения в журнале.

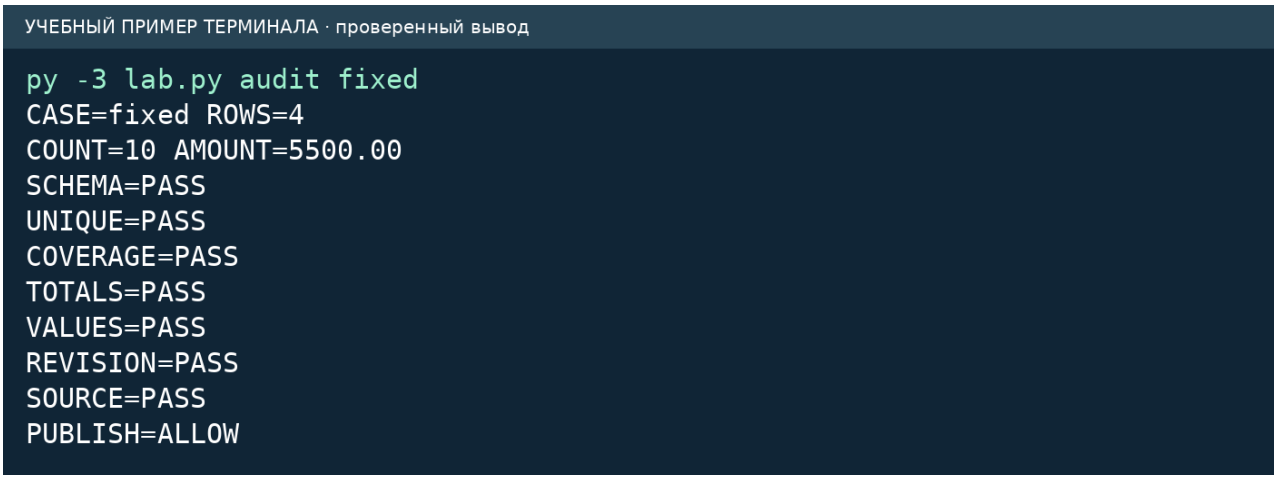
Ниже показан исходный дефект balanced, совпадающий с начальным fixed. В отчёте запишите изменения в форме «ключ — поле — было — стало — основание». Не исправляйте JSON и не заменяйте исходные примеры.

Что записать и проверить

После сохранения выполните audit fixed. Если VALUES остаётся FAIL, проверьте копейки, нужный файл и сохранение редактора.

12 Повторная проверка исправления

Выполните audit fixed или 09_AUDIT_FIXED.cmd. После правильного исправления все семь условий должны иметь PASS, PUBLISH=ALLOW. Откройте results/fixed.json и сверьте результат с ручным расчётом.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py audit fixed
```

Повторите ту же команду ещё раз без изменения файла. Результат должен сохраниться. Это повторяемость аудита неизменных данных, а не гарантия exactly-once обработки распределённой системой.

Для самопроверки прочитайте cases/fixed.csv Блокнотом: строки сумм в копейках — 70000, 150000, 230000, 100000. Общий итог остался 5500. Изменилось распределение, которое итоговая проверка ранее не замечала.

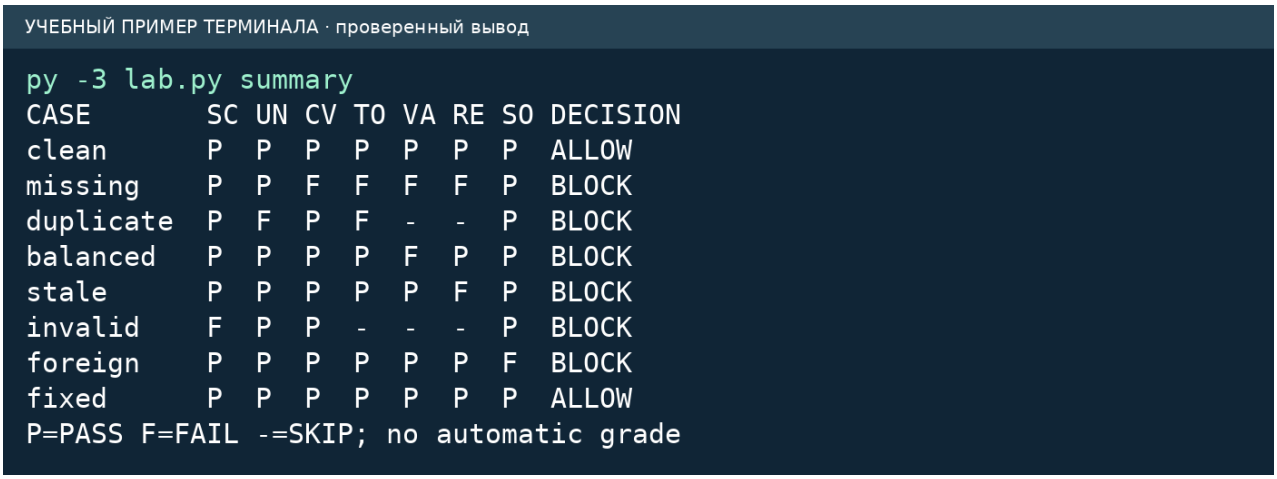
Если после аудита изменить CSV, отчёт станет устаревшим. Нужно снова запустить audit fixed. Программа упаковки не принимает протокол, не соответствующий текущим данным. Это помогает связать решение с конкретной версией файла.

Что записать и проверить

В пункт 5 отчёта внесите обе правки, основания, решение первого аудита и повторного запуска.

13 Общий протокол и собственный вывод

Запустите summary или 10_SUMMARY.cmd. Каждая строка относится к одному примеру. SC — схема; UN — уникальность; CV — покрытие; TO — итоги; VA — значения; RE — редакции; SO — источник. P=PASS, F=FAIL, дефис=SKIP.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py summary
```

NOT_RUN означает отсутствие аудита. Вернитесь к нужной команде. Сообщение Stale audit означает, что пример изменился после проверки: восстановите исходный пример либо повторите аудит рабочей копии. Не редактируйте JSON протокола вручную.

Перенесите решения в REPORT.txt своими словами, включая причины. Для clean и исправленного fixed ожидается ALLOW, для шести дефектных примеров — BLOCK. Объясните, почему число PASS не превращается в оценку и не позволяет голосовать за публикацию.

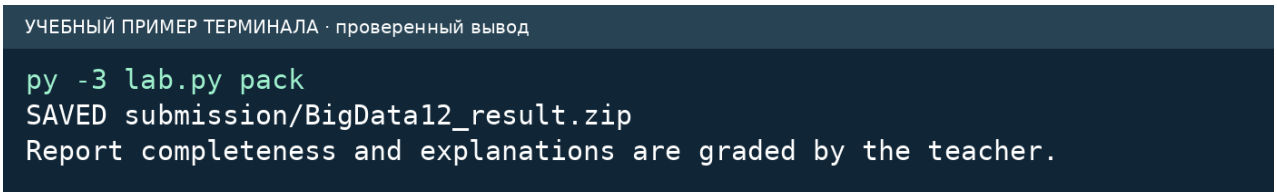
В выводе обязательно укажите область проверки: конечные 12 событий, один день, purchase/success, четыре окна и согласованный снимок. Здесь нет проверки неизвестных будущих событий, независимых серверов или надёжности внешнего хранилища.

Что записать и проверить

Контрольный вопрос: если добавить строку 10:20,0,0,1, общая сумма не изменится. Какая проверка обнаружит лишний ключ? Не меняйте файлы ради ответа.

14 Сохранение результата и сдача

Заполните все поля REPORT.txt. Не оставляйте [заполнить]; используйте собственные формулировки. Обязательно сохраните прогноз, ручной эталон, решения по всем примерам, обе правки и границы вывода.



Учебная иллюстрация команды и проверенного вывода

```
py -3 lab.py pack
```

Выполните pack или 11_PACK.cmd. Программа проверит актуальность восьми аудитов и ALLOW для fixed, затем создаст submission/BigData12_result.zip. Архив включает отчёт, источник, манифест, примеры и JSON протоколы. Исходный код сдавать не требуется.

Проверка заполненности отчёта техническая: она не определяет правильность ваших объяснений. Оценку выставляет преподаватель. Откройте созданный ZIP в Проводнике, убедитесь в наличии REPORT.txt и results/fixed.json. Сохраните архив до открытия задания в ЭОС.

При невозможности завершить работу сохраните папку целиком и запишите последний успешный этап. Не отправляйте пустой ZIP и не меняйте проверки программы ради упаковки. Срок завершения согласуется с преподавателем отдельно.

Что записать и проверить

В ЭОС позже прикрепите один BigData12_result.zip. Эта работа оценивается отдельно от предыдущей пары.

15 Оценивание и помощь при ошибках

Минимум для оценки 3: проверяемый ручной эталон; обнаружение пропуска, дубликата и компенсирующей ошибки; исправленный и проверенный fixed; объяснение различия запуска и качества. Полное выполнение включает все семь примеров и содержательный отчёт.

Оценка 4: все аудиты и исправление выполнены, но есть отдельные неточности в объяснениях или оформлении. Оценка 5: результаты и обоснования полны, верно разобраны SKIP, источник, редакции и ограничения. Оценка 2: минимум не достигнут; нужна доработка. Техническая упаковка сама по себе не гарантирует оценку.

Cannot open file lab.py: командная строка открыта не в папке проекта. Выйдите и снова введите cmd в адресной строке нужной папки. Source changed: источник отличается от манифеста; распакуйте чистую копию в новую папку, сохранив старую для разбора.

Missing or stale audit: выполните отсутствующую проверку или повторите аудит после изменения. Fixed table is not ready: прочитайте details в results/fixed.json. Fill REPORT: заполните все поля и сохраните файл. PUBLISH=BLOCK у дефектного примера: это ожидаемое обнаружение нарушения.

Методическая основа: предыдущий практикум о корректировке и раздел 11–12 РПД. Технические справки: docs.python.org/3/library/csv.html и docs.python.org/3/library/decimal.html. Примеры, контракт проверок и программа подготовлены для данного занятия.

Что записать и проверить

Проверьте себя: почему 5500 недостаточно? Почему четыре строки недостаточно? Почему один хеш недостаточен для доказательства реальной полноты?